

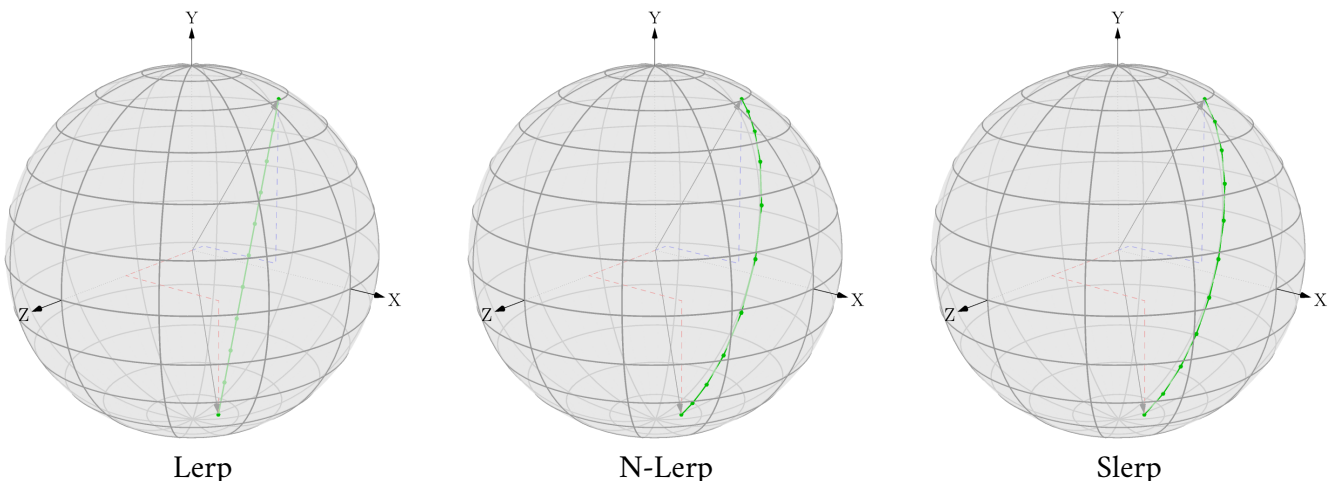
Kurz & Knapp

- Was ist der Unterschied von Lerp zu Nlerp bei Einheitsrichtungen? Was wird mit Nlerp erreicht? Was nicht?
- Was ist der Unterschied von Nlerp zu Slerp bei Einheitsrichtungen? Welchen Vorteil hat Slerp?
- Welche Komponenten gehen in eine homogene Transformationsmatrix ein, wie sieht eine solche für 5 dimensionale Koordinaten aus?
- Welche Daten sind zur Bestimmung der Eyespace Matrix in 3D nötig?
- Was ist der 'Up-Vector' und wozu ist er nötig?
- Was bedeuten n und f in den Projektionsmatrizen?
- Was ist der Unterschied einer orthographischen Projektion zu einer perspektivischen?
- Woran kann man sehr schnell erkennen, ob es sich bei einer Projektionsmatrix um eine orthographische oder perspektivische Projektion handelt?
- Wie werden aus dem Field-of-View Winkel und dem Seitenverhältnis der Kamera die Parameter l, r, t, b für die perspektivische Projektionsmatrix bestimmt?
- Die Parameter der Kamera gehen nun in die View-Matrix und die Projektionsmatrix ein, eine übliche Unterteilung ist in *intrinsische* und *extrinsische* Parameter. Welche der Parameter verorten Sie wo?

Aufgabe 1

Das im Aufgaben-Tarball enthaltene Asymptote Programm `slerp.asy` zeichnet eine dreidimensionale Kugel wie sie auch in den Vorlesungsfolien zu Slerp zu sehen ist. Das Programm berechnet verschiedene interpolierte Positionen zwischen den beiden Punkten auf der Kugel, wahlweise mit linearer Interpolation (siehe `lerp`, schon implementiert), mittels N-Lerp (siehe `nlerp`) und mittels Slerp (siehe `slerp`), wobei die letzten beiden zu implementieren sind. Vergleichen Sie die Ergebnisse.

Hier als Preview:



Hinweis: In diesem Fall werden Png-Bilder über Asymptotes Rasterisierer erzeugt, der entsprechende Aufruf für Asymptote ist dann `asy -f png -render 2 slerp.asy`. Der Aufruf ist auch über das mitgelieferte Makefile verfügbar. Schönere Ergebnisse erhalten Sie mit höheren Werten für `-render`.

Hinweis: Die 3D Darstellung in Asymptote ist nicht überall schön unterstützt. Mein Test auf halbwegs aktuellem Gentoo und Ubuntu 20.04.4 hat jedenfalls funktioniert, wenn Sie für die `lerp`-Variante keine Ausgabe produzieren können, dann liegt wahrscheinlich eine inkompatible Version vor.

Aufgabe 2

Im Tarball zu diesem Blatt findet sich das Asymptote Programm `eyespace.asy`, das neben einem kanonischen Koordinatensystem als Weltkoordinaten einen Eyespace und NDC in 2D definiert. Das Programm arbeitet nicht (nur) mit Asymptotes eingebauten Transformationen, sondern benutzt eigendefinierte homogene Transformationsmatrizen und Vektoren um den in der VL beschriebenen Ablauf möglichst genau abzubilden (siehe `homo2d.asy`).

Beachten Sie dabei, dass es sich um 2D homogene Daten handelt, d.h. die Räume sind zweidimensional und die 'Bildfläche' auf die projiziert wird ist entsprechend eindimensional.

In dieser Aufgabe ist nichts zu implementieren. *Aufgabe:* Lesen Sie bitte das Programm und stellen Sie sicher, dass Sie verstehen, wie und warum die Ausgabe so zustande kommt.

Aufgabe 3

Mit der orthographischen Projektionsmatrix P werden Positionen vom Eyespace in den NDC überführt, mit der Viewportmatrix W werden Positionen vom NDC auf Viewport- (aka Fenster-) Koordinaten abgebildet.

$$P = \begin{pmatrix} \frac{2}{x_{\max} - x_{\min}} & 0 & 0 & -\frac{x_{\max} + x_{\min}}{x_{\max} - x_{\min}} \\ 0 & \frac{2}{y_{\max} - y_{\min}} & 0 & -\frac{y_{\max} + y_{\min}}{y_{\max} - y_{\min}} \\ 0 & 0 & \frac{2}{n - f} & -\frac{f + n}{f - n} \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad W = \begin{pmatrix} \frac{w}{2} & 0 & 0 & x_{vp} + \frac{w}{2} \\ 0 & \frac{h}{2} & 0 & y_{vp} + \frac{h}{2} \\ 0 & 0 & \frac{f - n}{2} & \frac{f + n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Stellen Sie die Matrix P für $x_{\min} = -10$, $y_{\min} = -5$, $x_{\max} = 10$, $y_{\max} = 5$, $n = 1$, $f = 101$ auf und transformieren Sie ein paar Punkte im Eyespace nach NDC. Mit der Viewport-Matrix können Sie die Punkte dann auf eine beliebige Fenstergröße skalieren (z.B. 1080p in Vollbild: $x_{vp} = y_{vp} = 0$, $w = 1920$, $h = 1080$).

Definieren Sie sich zwei Dreiecke (die sich nach der Projektion überlappen) und verifizieren Sie, dass (a) keine Verzerrung oder Größenänderung durch Abstand vom Kameraursprung stattfindet und (b) dass Sie die Dreiecksteile die je ein Pixel überlappen in der Tiefe (Entfernung zur Kamera) ordnen können.

Als Übung zur Rasterisierung können Sie die Dreiecke auf einem Pixelgitter (gerne auf einem kleineren als zuvor vorgeschlagen) rastern und mittels Baryzentrischer Interpolation die Tiefe pro Pixel bestimmen. Ein Pixel wird nur dann von einem Dreieck eingefärbt, wenn für dieses Pixel noch kein näheres Dreieck bearbeitet wurde (wie können Sie das systematisch festhalten?).

Das können Sie einfach mit Stift und Papier machen, oder z.B. mit Asymptote implementieren.

Aufgabe 4

Gegeben eine Transformation mit einer Projektionsmatrix

$$P = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{l+r}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{b+t}{t-b} & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad \mathbf{v}' = P\mathbf{v},$$

bestimmen Sie den maximalen Wert von x der in NDC auf 1 abgebildet wird, abhängig von z , also die Funktion $x_{\max}(z)$. Berechnen Sie dann diese Funktion für $z = -n$ und $z = -f$.

Hinweis: Die Parameter n und f werden als Absolutbetrag angegeben und nur für reguläre z -Werte negiert verwendet.

Aufgabe 5

Im `Tarball renderpipeline.tar.bz2` finden Sie eine kleine C++ Implementierung einer Renderpipeline mit Rasterisierung wie bisher in der Vorlesung besprochen. Zum Übersetzen ist es nötig `libpng++` und `glm` zu installieren, auf Debian-Derivaten sind die Pakete einfach verfügbar:

```
apt install libpng++ libglm-dev
```

Das Programm verarbeitet Eingaben auf der Kommandozeile, erzeugt ein Dreiecksnetz, wendet Transformationen auf die Dreiecke des Netzes an, rasterisiert die so transformierten Dreiecke und schreibt am Ende ein Bild ins Dateisystem. Zur besseren Darstellung wird jedes Dreieck mit einer zufälligen Farbe gezeichnet. Diesen Ablauf können Sie in der Datei `main.cpp` ablesen.

Die Kommandozeilenschnittstelle ist in `cmdline.cpp` definiert, die entsprechenden Variablen und Initialwerte finden Sie in `cmdline.h`. Diese Dateien müssen nicht verändert werden.

In `raster.cpp` finden Sie eine Implementierung der SD-Rasterisierung wie sie im ersten Blatt mit Asymptote als Aufgabe gestellt war. Lesen Sie den Code durch und prüfen Sie, ob Sie alles nachvollziehen können. Der einzig neue Teil sollte die Verwendung eines sog. *Framebuffers* sein, der als Ausgabe des Renderns verwendet wird. Dabei handelt es sich in unserem Fall nur um ein im Speicher vorgehaltenes PNG Bild, das später gespeichert wird.

Informationen zum Aufruf des Programms können Sie sich mittels `./renderpipeline --help` schön aufbereitet ansehen.



Aufgabe 5.1

Beginnen Sie damit, ein Dreieck in NDC zu rendern. Die Standardwerte der Kommandozeilenparameter sind für diesen Fall eingestellt, Sie können diesen Modus explizit mittels `-s ndc-tri` (bzw. `--scene ndc-tri`) angeben. In der Szenenauswahl wird dann der erste Zweig ausgeführt (`scene.cpp:9`), in dem bisher ein Platzhalter-Dreieck erzeugt wird. Überlegen Sie hier was gültige NDC-Koordinaten für ein Dreieck sein könnten, wie es aussehen sollte und tragen Sie die Koordinaten ein. Wird auf der Kommandozeile explizit ein z -Wert angegeben (z.B. `-Z 0.5`), so soll dieser verwendet werden.

Wenn Sie das Programm nun ausführen werden Sie wahrscheinlich kein Ergebnis sehen. Was noch fehlt ist die Transformation des NDC-Dreiecks in Window-Koordinaten, wie es der Rasterisierer erwartet. Erweitern Sie dazu die Funktion `window_transform` in `main.cpp` und füllen Sie den Inhalt von `W_transform`. In der Abbildung oben links sehen Sie ein *beispielhaftes* Ergebnis mit `-w 192 -h 108` (das natürlich von Ihrem konkret gewählten Dreieck abweichen kann).

Hinweise.

- GLM Matrizen werden in Spalten-Ordnung geschrieben, d.h. insbesondere beim Erzeugen einer neuen Matrix werden Vektoren angegeben, die als separate Spalten der Matrix zu verstehen sind. Der Hinweis steht hier prominent, da die Matrizen im Code deshalb transponiert stehen.

- Sie können Vektoren und Matrizen aus GLM mittels `cout << to_string(...) << endl;` ausgeben.
- Zum Debuggen können Sie im Rasterisierer `overlaps_pixel` anpassen, z.B. um nur die Bounding Box zu prüfen.
- Beachten Sie, dass in der Rasterisierung erwartet wird, dass die Dreieckskanten gegen den Uhrzeigersinn verlaufen.
- Aus einem `vec3 v` können Sie via `vec4(v, 1.0f)` einen homogenen Vektor erzeugen, aus einem `vec4 w` via `v = w` einen kartesischen. Aber: Hier wird nicht implizit dehomogenisiert, es wird einfach die letzte Komponente verworfen!

Als Ausgabe wird standardmäßig die Datei `out.png` angelegt, der Hintergrund ist dunkelgrau, das Dreieck in einer zufälligen Farbe. In der Abbildung oben sehen Sie in der Mitte ein beispielhaftes Ergebnis. Prüfen Sie was passiert, wenn Sie z im gültigen Bereich verschieben. Was passiert und warum?

Aufgabe 5.2

Verwenden Sie den Kommandozeilenschalter `-s es-tri`. Nun soll als 'Szene' ein Dreieck im Eye Space erzeugt werden. Gehen Sie wie in der vorigen Aufgabe beschrieben vor.

Passen Sie weiterhin die Funktion `perspective_projection_transform` an und verwenden Sie diese korrekt in `PW_transform`. Achten Sie darauf, dass der Rasterisierer nach wie vor Eingangsdaten in Window-Koordinaten erwartet.

Prüfen Sie auch hier, was passiert wenn Sie z im gültigen Bereich verschieben. Was passiert und warum?

Aufgabe 5.3

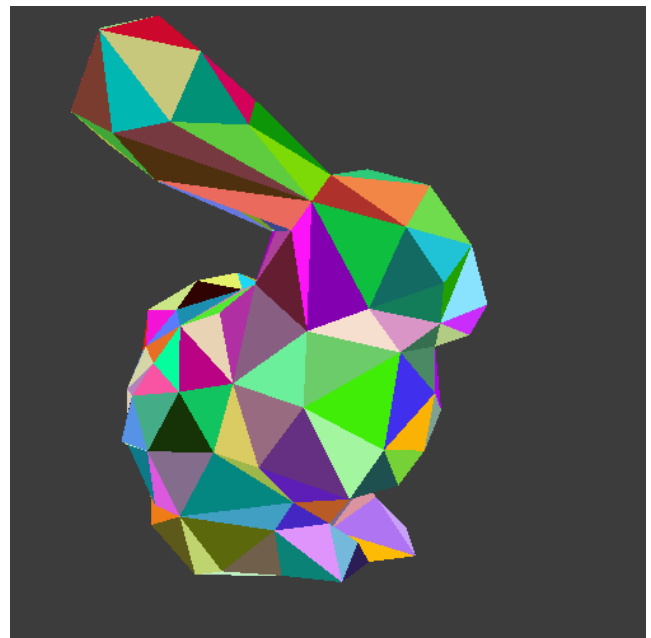
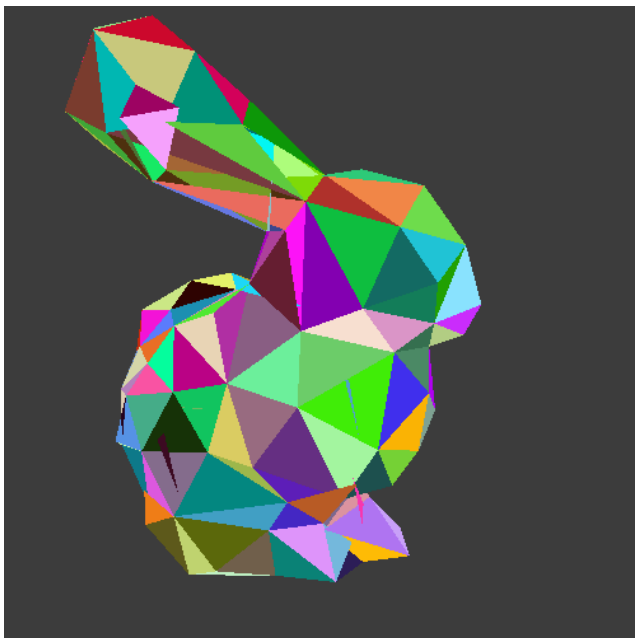
Verwenden Sie nun den Kommandozeilenschalter `-s bunny`. Damit wird ein größeres Mesh das in Weltkoordinaten angegeben ist geladen. An `scene.cpp` ist in diesem Fall keine Änderung nötig.

Die Eye Space Parameter Kameraposition ($-p$), Blick-Richtung ($-d$) und Up-Vektor ($-u$) können alle auf der Kommandozeile angegeben werden und haben als Standardeinstellung den $(0, 0, 0)^T$, $(0, 0, -1)^T$ und $(0, 1, 0)^T$.

Implementieren Sie `viewing_transform` und `VPW_transform` in `main.cpp` und prüfen Sie, ob Sie an Position $(-75, 75, 60)^T$ mit Blickrichtung $(1, -1, 0)^T$ und Up-Vektor $(0, 0, 1)^T$ das selbe Ergebnis erhalten, wie in der Abbildung oben rechts.

Mit dem beiliegenden Python-Skript können Sie (sofern Sie `ffmpeg` installiert haben) eine Animation um das Modell als Video rendern. Ihre Lösung sollte zum Beispiel-Video passen.

Aufgabe 6



Erweitern Sie Ihr Programm aus dem vorigen Aufgabe so, dass neben dem Framebuffer auch ein Tiefenpuffer gehalten, und beim Rasterisieren geprüft wird, ob das aktuelle Fragment näher an der Kamera ist als das schon im Framebuffer befindliche. Dazu müssen Sie sich etwas in den Code in `render.cpp` einlesen und diesen so erweitern, dass Sie Tiefenwerte speichern und zu einer Position auf einem Dreieck einen korrekten Tiefenwert bestimmen können.

Überlegen Sie (oder rechnen Sie durch) welchen Wertebereich die Tiefenwerte nach der Transformationspipeline in der Aufgabe beim Rasterisieren haben und initialisieren Sie den Tiefenpuffer passend.

Das Beispielbild oben zeigt die Konfiguration

`$ ./renderpipeline -s bunny -p -75,75,60 -d 1,-1,0 -u 0,0,1 -w 500 -h 500`
in der ursprünglichen Variante (links) und mit Tiefenpuffer (rechts).

Wie Sie diese Aufgabe erst nach der nächsten Vorlesung mit weiteren Details zur z -Kurve bearbeiten ist das auch ok :)

Happy Hacking :)